

Modül 05 — Alıştırmalar

Veri işleme: dplyr ve tidyr

Prof. Dr. Hakan Mehmetcik

2026-09-11

Bu Alıştırmalar Hakkında

Bu alıştırmalar **notlandırılmaz ve teslim edilmez**. Amacı, sınavdan önce kendi durumunuzu ölçebilmenizdir. Sınav soruları da bu biçimde (çoktan seçmeli, tek doğru cevap) kurgulanır.

Her soru 4 şıklıdır ve yalnızca **bir tanesi** doğrudur. Sınavda bilgisayar olmayacak; önce kâğıt üzerinde düşünün, R’da doğrulamayı sonraya bırakın.

Cevaplarınızı not edin, sonra 05_modul_cevap_anahtari.qmd dosyasıyla karşılaştırın.

Bölüm A — select(), filter(), arrange()

1. `gapminder |> select(country, year)` kodu ne yapar?

- a) Yalnızca `country` ve `year` sütunlarını tutar, diğer tüm sütunları atar
- b) Yalnızca `country` ve `year` değerlerine sahip satırları tutar
- c) Veri setini `country` ve `year`’a göre sıralar
- d) Veri setinden yalnızca `country` ve `year` sütunlarını siler

2. Aşağıdaki kod **sıfır satır** döndürüyor. Sorun nedir?

```
gapminder |> filter(continent == "Europe", continent == "Asia")
```

- a) `filter()` iki koşulu aynı anda kabul etmez
- b) Virgülle ayrılan koşullar **VE** (&) anlamına gelir; bir ülke aynı anda hem Avrupa hem Asya kıtasında olamayacağı için hiçbir satır her iki koşulu birden sağlamaz — istenen muhtemelen **VEYA** (|)

- c) `continent` sütunu sayısal olmadığı için `filter()` çalışmaz
- d) `gapminder` veri setinde `continent` sütunu yoktur

3. `gapminder |> arrange(gdpPercap)` çalıştırıldığında ilk satırda hangi ülke görünür?

- a) Kişi başı GSYİH'si **en yüksek** olan ülke-yıl
- b) Kişi başı GSYİH'si **en düşük** olan ülke-yıl — `arrange()` varsayılan olarak **artan** sırada sıralar
- c) Alfabetik olarak ilk sıradaki ülke
- d) Rastgele bir ülke; `arrange()` sıralama garantisi vermez

4. `filter(continent == "Africa" | continent == "Asia")` ile `filter(continent %in% c("Africa", "Asia"))` arasındaki ilişki nedir?

- a) Farklı sonuç üretirler; `%in%` yalnızca sayısal sütunlarda çalışır
- b) Aynı sonucu üretirler; ikisi de “Afrika **veya** Asya” kıtasındaki satırları tutar, `%in%` çok sayıda seçenek olduğunda daha okunaklıdır
- c) `%in%` yalnızca tek bir değerle karşılaştırma yapabilir
- d) `|` operatörü R'da tanımlı değildir

5. Aşağıdaki zincir çalıştırıldığında elde edilen tablo kaç sütun içerir?

```
gapminder |>
  filter(year == 2007) |>
  select(country, lifeExp) |>
  arrange(desc(lifeExp))
```

- a) 6 (`gapminder`'ın tüm sütunları)
- b) 2 (`country` ve `lifeExp`) — `filter()` ve `arrange()` satırları etkiler, sütun sayısını değiştirmez; sütun sayısını yalnızca `select()` belirler
- c) 1, çünkü `arrange()` bir sütunu sıralama anahtarı olarak “tüketir”
- d) 0, çünkü `year == 2007` koşulu `select()`'ten sonra geçersizleşir

Bölüm B — `mutate()`, `group_by()` + `summarize()`

6. `mutate()` ile `rename()` arasındaki temel fark nedir?

- a) İkisi de aynı işi yapar, yalnızca isimleri farklıdır
- b) `mutate()` yeni bir sütun **türetir** (değerleri hesaplar); `rename()` mevcut bir sütunun yalnızca **adını** değiştirir, değerlerine dokunmaz
- c) `rename()` yalnızca sayısal sütunlarda çalışır
- d) `mutate()` satır sayısını değiştirir, `rename()` değiştirmez

7. Aşağıdaki kod çalıştırıldığında `oran` sütunu her satırda **1.0** çıkıyor. Sorun nedir?

```
gapminder |>
  filter(year == 2007) |>
  mutate(oran = pop / sum(pop))
```

- a) `mutate()` toplamla bölme işlemini desteklemez
- b) `sum(pop)` **tüm dünyanın** nüfus toplamını değil, gruplanmamış tüm satırların toplamını almalıydı zaten — burada `group_by(continent)` eksik olduğu için her satır kendi kıtasının payına değil, dünya toplamına bölünüyor olması **beklenirdi**; sonucun 1.0 çıkması, kodun `summarise()` ile karıştırıldığını, aslında `mutate()`'in satır sayısını koruyarak her satıra dünya toplamını yaydığını gösterir
- c) `pop` sütunu metin (character) olarak kaydedilmiştir
- d) `year == 2007` filtresi hiçbir satır bırakmamıştır

8. `gapminder |> group_by(continent) |> summarise(m = mean(lifeExp))` kodunun çıktısında kaç satır olur?

- a) 142 (gapminder'daki ülke sayısı kadar)
- b) 5 — `continent` sütununda kaç farklı kıta varsa (Africa, Americas, Asia, Europe, Oceania) o kadar satır; `summarise()` her grubu tek satıra indirger
- c) 1, çünkü `summarise()` her zaman tüm veriyi tek satıra indirger
- d) 12, çünkü veri setinde 12 farklı yıl vardır

9. `group_by()` sonrası `summarise()` **kullanmadan** doğrudan `arrange()` çağırırsanız ne olur?

- a) Hata alırsınız; `group_by()` yalnızca `summarise()` ile birlikte kullanılabilir
- b) Veri, gruplanmış hâliyle ama satır sayısı **değişmeden** sıralanır — `group_by()` tek başına veriyi özetlemez, yalnızca sonraki işlemlerin grup grup uygulanacağını işaretler
- c) Yalnızca ilk grup görüntülenir, diğerleri atılır
- d) Sütunlar alfabetik sıraya göre yeniden düzenlenir

10. Aşağıdaki zincir hangi soruya cevap verir?

```
gapminder |>
  filter(year == 2007, continent == "Asia", pop > 10e6) |>
  arrange(desc(lifeExp)) |>
  select(country, lifeExp) |>
  head(3)
```

- a) 2007'de dünyadaki en kalabalık 3 ülke hangileri?
- b) 2007'de, nüfusu 10 milyonun üzerindeki Asya ülkeleri arasında yaşam beklentisi en yüksek 3 ülke hangileri?
- c) Asya kıtasındaki tüm ülkelerin ortalama yaşam beklentisi kaçtır?
- d) Hangi Asya ülkelerinin nüfusu tam olarak 10 milyondur?

Bölüm C — Düzenli (Tidy) Veri, `pivot_longer()`, `pivot_wider()`

11. Aşağıdakilerden hangisi düzenli (tidy) verinin **üç kuralından biri değildir**?

- a) Her değişken kendi sütununu oluşturur
- b) Her gözlem kendi satırını oluşturur
- c) Her veri seti en fazla 10 sütun içermelidir
- d) Her değer kendi hücrelerini oluşturur

12. Bir tabloda yıllar (1952, 1957, ... 2007) ayrı sütunlar olarak yer alıyorsa, bu tablo neden tidy **değildir**?

- a) Çünkü sütun adları sayıyla başlamaktadır
- b) Çünkü “yıl” aslında bir **değişkendir**, ama burada bir değişken olarak bir sütuna değil, 12 ayrı sütun **adına** dağıtılmıştır — bu, “her değişken kendi sütununu oluşturur” kuralını ihlal eder
- c) Çünkü tablo çok fazla satır içermektedir
- d) Geniş formatlı hiçbir tablo R’da okunamaz

13. `pivot_longer(cols = \1952:‘2007’, names_to = “yıl”, values_to = “yaşam”)` çalıştırdınız ama sonuçta `yıl` sütununda yaşam beklentisi sayıları, `yaşam` sütununda ise yıllar görünüyor. Hata nedir?

- a) `names_to` ve `values_to` argümanlarının yerleri karışmış: `names_to`, eski **sütun adlarının** (yılların) gideceği yeni sütunu; `values_to` ise eski **hücre değerlerinin** (yaşam beklentisi sayılarının) gideceği yeni sütunu belirtir
- b) `pivot_longer()` fonksiyonu R’da mevcut değildir
- c) `cols` argümanına yanlış veri türü girilmiştir
- d) Bu davranış normaldir, `pivot_longer()` her zaman sütunları karıştırır

14. `pivot_wider(names_from = year, values_from = ort)` kodu ne yapar?

- a) `year` sütunundaki **değerleri** yeni sütun adlarına dönüştürür, `ort` sütunundaki değerleri de bu yeni sütunların içeriği yapar — `pivot_longer()`’ın tam tersi
- b) `year` ve `ort` sütunlarını siler
- c) Veriyi `year`’a göre alfabetik sıralar
- d) Yalnızca `year` sütununu yeniden adlandırır

15. Bir analiz raporunda “kıtaların 1987, 1997 ve 2007’deki ortalama yaşam beklentisini tek bir tabloda, yılları sütun olarak” göstermek istiyorsunuz. `group_by()` + `summarise()` sonrası hangi fonksiyonu kullanmalısınız?

- a) `pivot_longer()`, çünkü rapor tabloları her zaman uzun formatta olmalıdır
- b) `pivot_wider()`, çünkü hedef biçim (yıllar sütunlarda) geniş formattır — analiz için uzun format tercih edilirken, sunum/rapor için geniş format genellikle daha okunaklıdır
- c) `arrange()`, çünkü yalnızca sıralama gerekir
- d) `rename()`, çünkü yalnızca sütun adları değişecektir

Bölüm D — Birleştirmeler (Joins)

16. `orgutler` (4 satır, örgüt bilgisi) ile `uyelikler` (60 satır, ülke-örgüt eşleşmesi) tablolarını `inner_join(uyelikler, orgutler, by = "orgut_kodu")` ile birleştirdiğinizde kaç satır elde edersiniz?

- a) 4, çünkü `orgutler` 4 satır içerir
- b) 64, iki tablonun satır sayısı toplanır
- c) 60 — `uyelikler`'deki her satır, `orgutler`'de eşleşen tek bir örgüt bilgisiyle **genişletilir**; satır sayısı değişmez, yalnızca yeni sütunlar (`orgut_adi`, `kurulus_yili`) eklenir
- d) 240, her ülke her örgütle eşleştirilir

17. `gm2007 |> left_join(uyelikler, by = c("country" = "ulke"))` kodunda, hiçbir örgüte üye olmayan bir ülkenin `orgut_kodu` sütununda ne görünür?

- a) O ülke tabloda tamamen kaybolur
- b) NA — `left_join()`, sol tablodaki (`gm2007`) **tüm satırları** korur; sağ tabloda eşleşme yoksa yalnızca o sütunlar boş (NA) kalır
- c) 0 (sıfır)
- d) Bir hata mesajı; `left_join()` eşleşmeyen satırlara izin vermez

18. `semi_join(x, y)` ile `inner_join(x, y)` arasındaki temel fark nedir?

- a) Aynı işi yaparlar, yalnızca isimleri farklıdır
- b) `semi_join()`, `y`'de eşleşmesi olan `x` satırlarını **olduğu gibi** tutar ve `y`'den hiçbir sütun eklemeyiz; `inner_join()` ise eşleşen satırları tutarken `y`'nin sütunlarını da ekler
- c) `semi_join()` yalnızca sayısal sütunlarda çalışır
- d) `inner_join()` satır sayısını asla değiştirmez, `semi_join()` değiştirir

19. `anti_join()` bir araştırmacı için pratikte en çok ne için kullanışlıdır?

- a) İki tabloyu birleştirip yeni sütunlar eklemek için
- b) Veri setindeki **kapsam boşluklarını** görmek için — ör. bir listede olup diğerinde karşılığı olmayan gözlemleri (ülkeleri, kayıtları) hızlıca ortaya çıkarmak
- c) Bir tabloyu alfabetik olarak sıralamak için
- d) Sayısal bir sütunun ortalamasını hesaplamak için

20. `gapminder$country` sütunu ile `uyelikler$ulke` sütununu birleştirmek için `by = c("country" = "ulke")` yazmamızın sebebi nedir?

- a) İki sütunun veri türü farklı olduğu için özel bir dönüşüm gerekir
- b) İki tabloda **aynı bilgiyi tutan sütunların adları farklıdır** (`country` vs. `ulke`); `by = c("sol_ad" = "sag_ad")` sözdizimi, `join` fonksiyonuna hangi sütunun hangisiyle eşleştirileceğini açıkça söyler
- c) Bu yazım yalnızca `anti_join()` için geçerlidir
- d) `by` argümanı olmadan hiçbir `join` fonksiyonu çalışmaz